

Shared ViewModel Across Fragments

1. Before you begin
2. Starter app overview
3. Complete the Navigation Graph
4. Create a shared ViewModel
5. Use the ViewModel to update the UI
6. Use ViewModel with data binding
7. Update pickup and summary fragment to use view model
8. Calculate price from order details
9. Setup click listeners using listener binding
10. Solution code
11. Summary
12. Learn more

1. Before you begin

You have learned how to use activities, fragments, intents, data binding, navigation components, and the basics of architecture components. In this codelab, you will put everything together and work on an advanced sample, a cupcake ordering app.

You will learn how to use a [shared ViewModel](#) to share data between the fragments of the same activity and new concepts like `LiveData` transformations.

Prerequisites

- Comfortable with reading and understanding Android layouts in XML
- Familiar with the basics of the [Jetpack Navigation](#) Component
- Able to create a navigation graph with fragment destinations in an app
- Have previously used fragments within an activity
- Can create a `ViewModel` to store app data
- Can use data binding with `LiveData` to keep the UI up-to-date with the app data in the `ViewModel`

What you'll learn

- How to implement recommended app architecture practices within a more advanced use case
- How to use a shared `ViewModel` across fragments in an activity

- How to apply a `LiveData` transformation

What you'll build

- A **Cupcake** app that displays an order flow for cupcakes, allowing the user to choose the cupcake flavor, quantity, and pickup date.

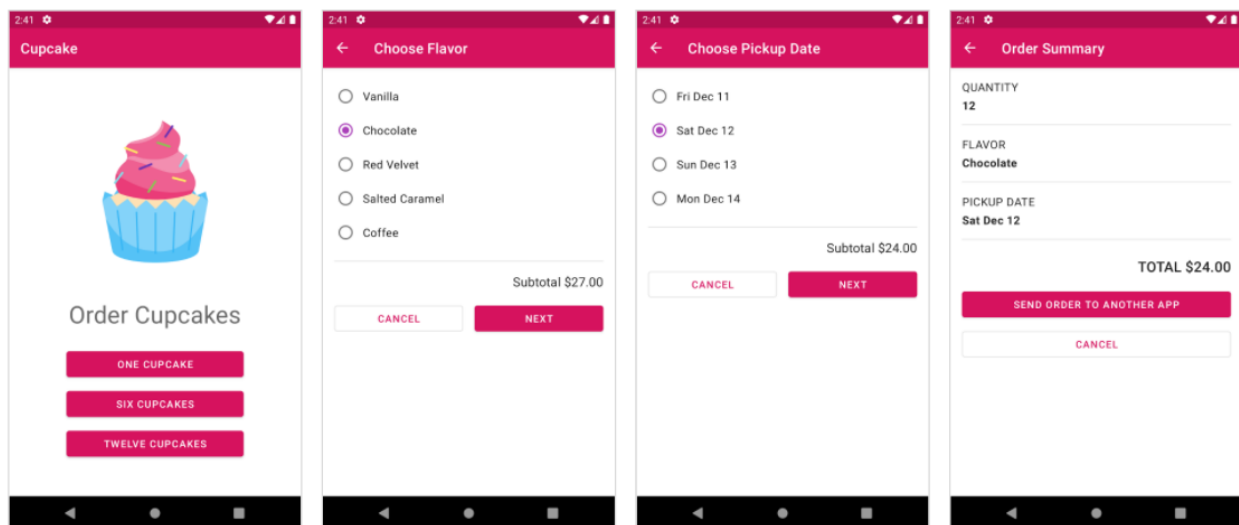
What you need

- A computer with Android Studio installed.
- [Starter code](#) for the **Cupcake** app.

2. Starter app overview

Cupcake app overview

The cupcake app demonstrates how to design and implement an online ordering app. At the end of this pathway, you will have completed the **Cupcake** app with the following screens. The user can choose the quantity, flavor, and other options for the cupcake order.



Download the starter code for this codelab

This codelab provides starter code for you to extend with features taught in this codelab. The starter code will contain code that is familiar to you from previous codelabs.

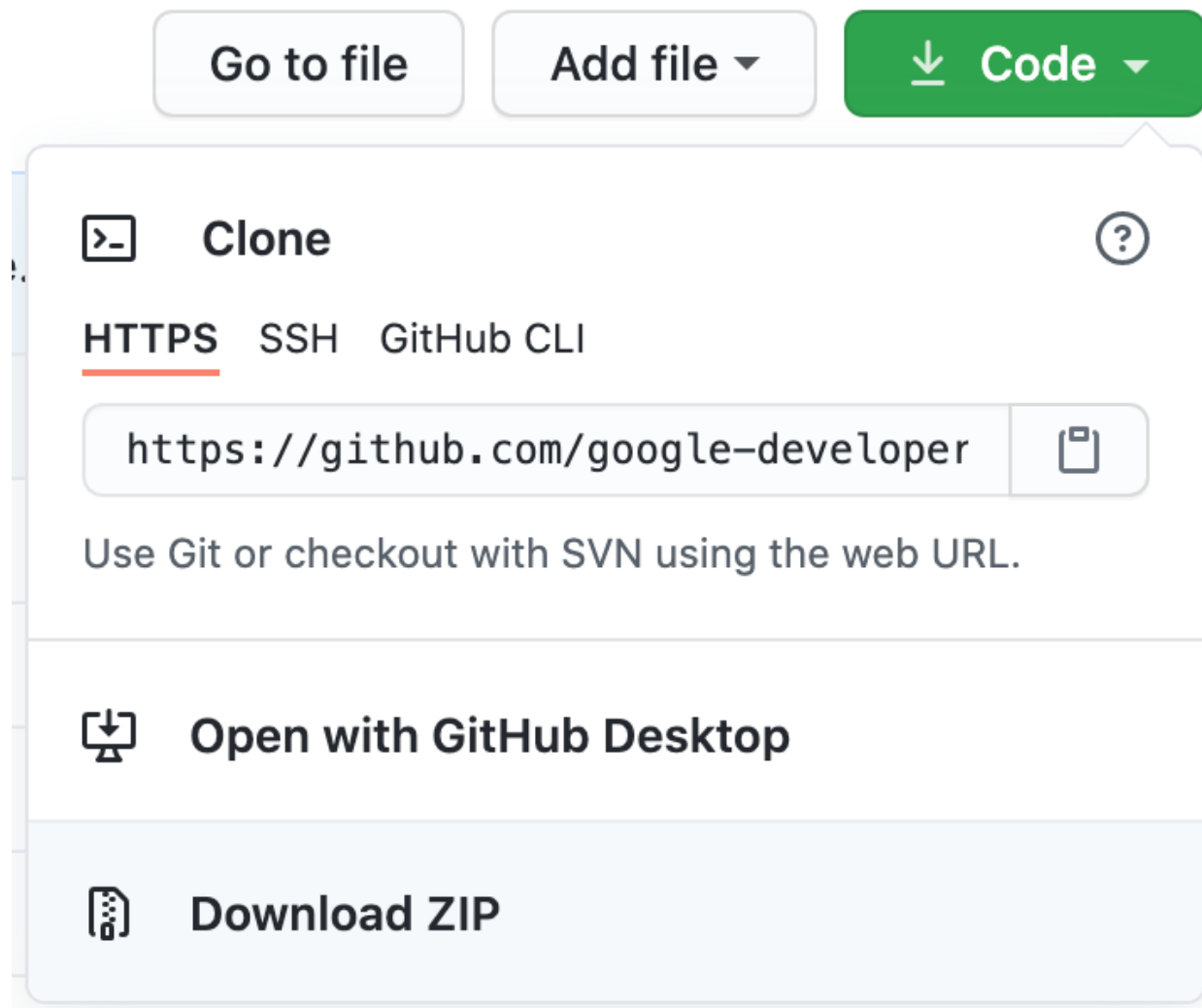
If you download the starter code from GitHub, note that the folder name of the project is `android-basics-kotlin-cupcake-app-starter`. Select this folder when you open the project in Android Studio.

Starter Code URL: <https://github.com/google-developer-training/android-basics-kotlin-cupcake-app/tree/starter>

To get the code for this codelab and open it in Android Studio, do the following.

Get the code

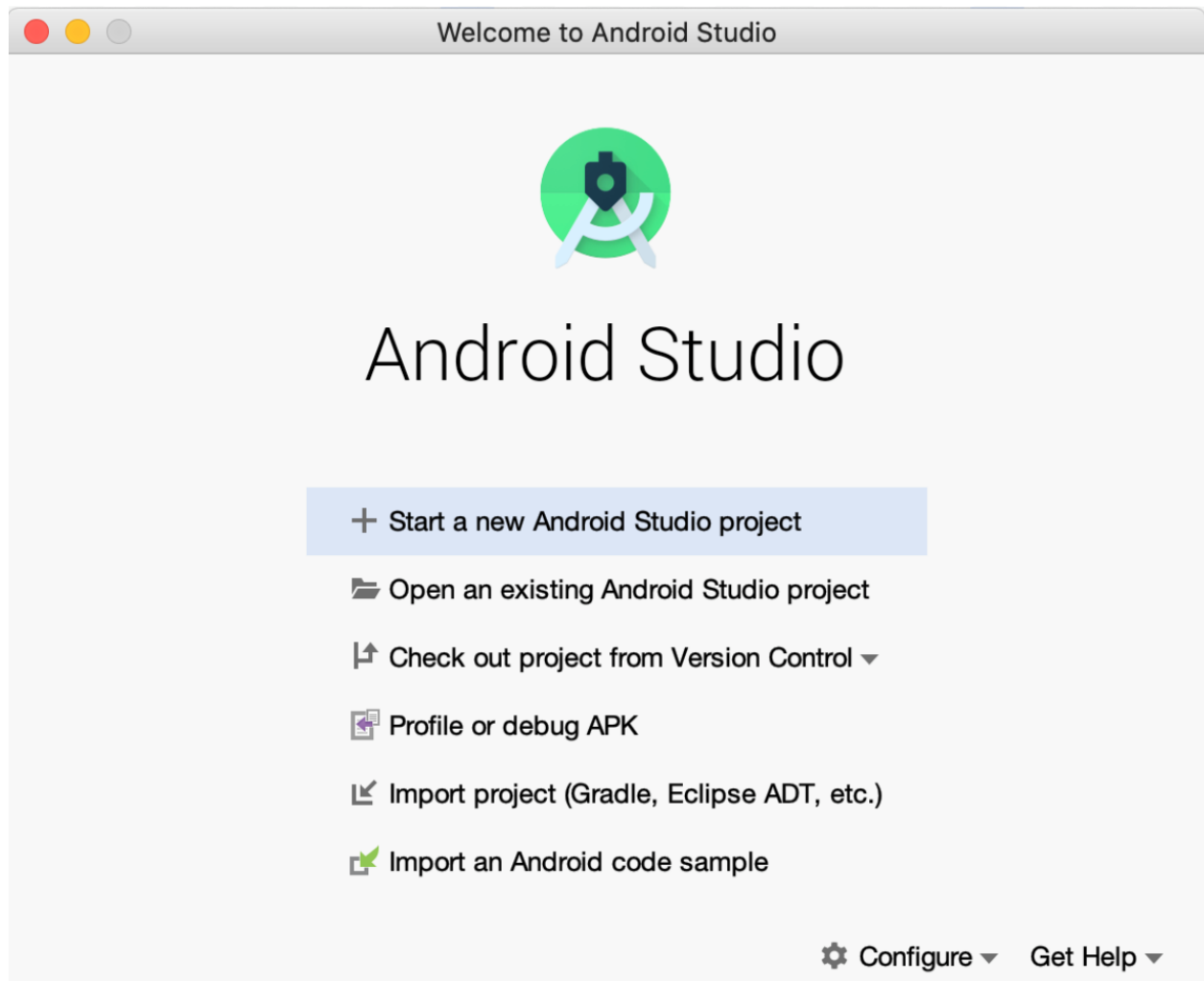
1. Click on the provided URL. This opens the GitHub page for the project in a browser.
2. On the GitHub page for the project, click the **Code** button, which brings up a dialog.



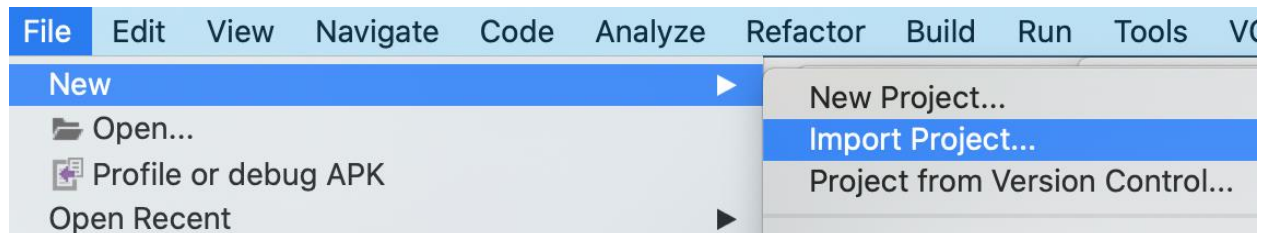
3. In the dialog, click the **Download ZIP** button to save the project to your computer. Wait for the download to complete.
4. Locate the file on your computer (likely in the **Downloads** folder).
5. Double-click the ZIP file to unpack it. This creates a new folder that contains the project files.

Open the project in Android Studio

1. Start Android Studio.
2. In the **Welcome to Android Studio** window, click **Open an existing Android Studio project**.



Note: If Android Studio is already open, instead, select the **File > New > Import Project** menu option.



3. In the **Import Project** dialog, navigate to where the unzipped project folder is located (likely in your **Downloads** folder).
4. Double-click on that project folder.
5. Wait for Android Studio to open the project.



6. Click the **Run** button to build and run the app. Make sure it builds as expected.
7. Browse the project files in the **Project** tool window to see how the app is set-up.

Starter code walk through

1. Open the downloaded project in Android Studio. The folder name of the project is `android-basics-kotlin-cupcake-app-starter`. Then run the app.
2. Browse the files to understand the starter code. For layout files, you can use the **Split** option in the top right corner to see a preview of the layout and the XML at the same time.
3. When you compile and run the app, you'll notice the app is incomplete. The buttons don't do much (except for displaying a `Toast` message) and you can't navigate to the other fragments.

Here's a walkthrough of important files in the project.

MainActivity:

The `MainActivity` has similar code to the default generated code, which sets the activity's content view as `activity_main.xml`. This code uses a parameterized constructor `AppCompatActivity(@LayoutRes int contentLayoutId)` which takes in a layout that will be inflated as part of `super.onCreate(savedInstanceState)`.

Code in the `MainActivity` class

```
class MainActivity : AppCompatActivity(R.layout.activity_main)
```

is same as the following code using the default `AppCompatActivity` constructor:

```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
    }  
}
```

Layouts (res/layout folder):

The `layout` resource folder contains activity and fragment layout files. These are simple layout files, and the XML is familiar from the previous codelabs.

- `fragment_start.xml` is the first screen shown in the app. It has a cupcake image and three buttons to choose the number of cupcakes to order: one cupcake, six cupcakes, and twelve cupcakes.
- `fragment_flavor.xml` shows a list of cupcake flavors as radio button options with a **Next** button.
- `fragment_pickup.xml` provides an option to select pickup day and a **Next** button to go to the summary screen.
- `fragment_summary.xml` displays a summary of the order details such as quantity, flavor and a button to send the order to another app.

Fragment classes:

- `StartFragment.kt` is the first screen shown in the app. This class contains the view binding code and a click handler for the three buttons.
- `FlavorFragment.kt`, `PickupFragment.kt`, and `SummaryFragment.kt` classes contain mostly boilerplate code and a click handler for the **Next** or **Send Order to Another App** button, which show a toast message.

Resources (res folder):

- `drawable` folder contains the cupcake asset for the first screen, as well as the launcher icon files.
- `navigation/nav_graph.xml` contains four fragment destinations (`startFragment`, `flavorFragment`, `pickupFragment`, and `summaryFragment`) without *Actions*, which you will define later in the codelab.
- `values` folder contains the colors, dimensions, strings, styles, and themes used for customizing the app theme. You should be familiar with these resource types from previous codelabs.

3. Complete the Navigation Graph

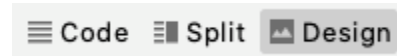
In this task, you'll connect the screens of the **Cupcake** app together and finish implementing proper navigation within the app.

Do you remember what we need to use the Navigation component? Follow [this guide](#) for a refresher on how to set up your project and app to:

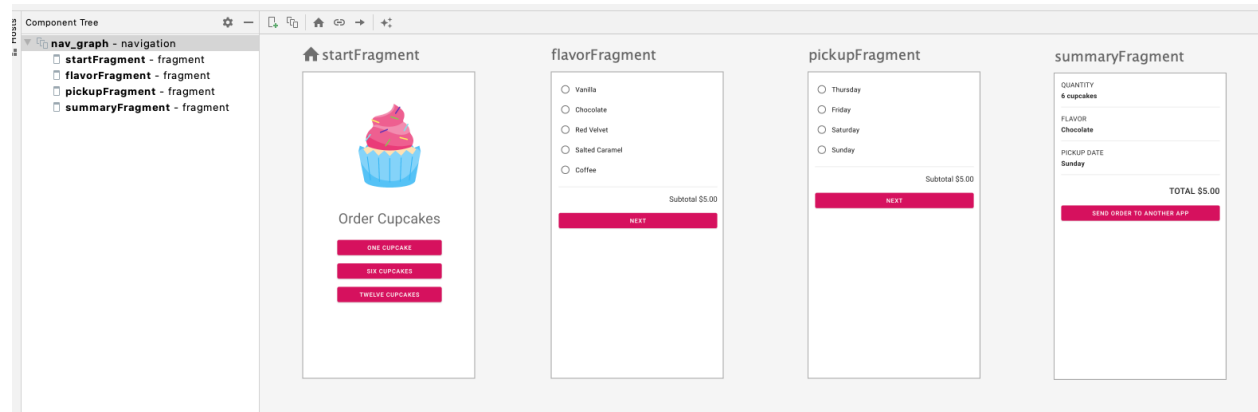
- Include the [Jetpack Navigation library](#)
- Add a `NavHost` to the activity
- Create a navigation graph
- Add fragment destinations to the navigation graph

Connect destinations in navigation graph

1. In Android Studio, in the **Project** window, open **res > navigation > nav_graph.xml** file. Switch to the **Design** tab, if it's not already selected.

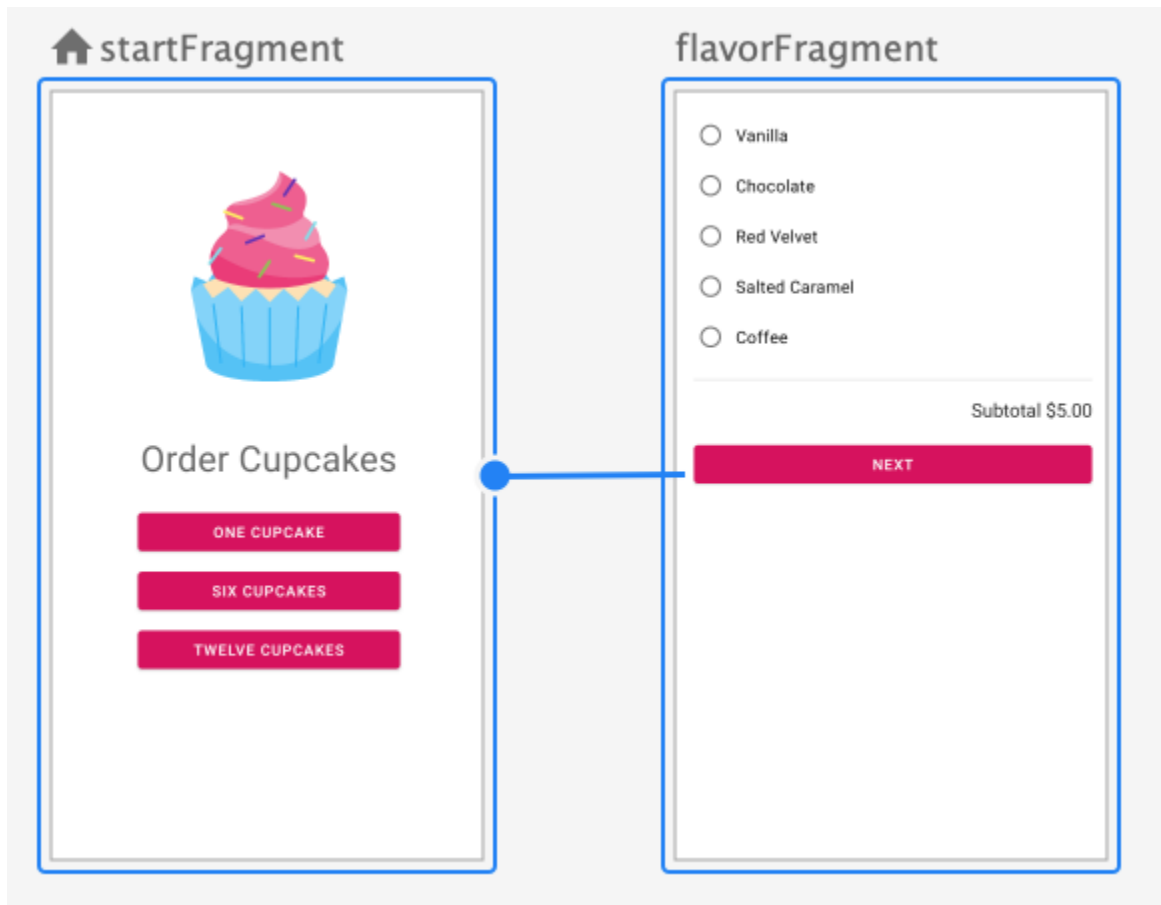


2. This opens the **Navigation Editor** to visualize the navigation graph in your app. You should see the four fragments that already exist in the app.

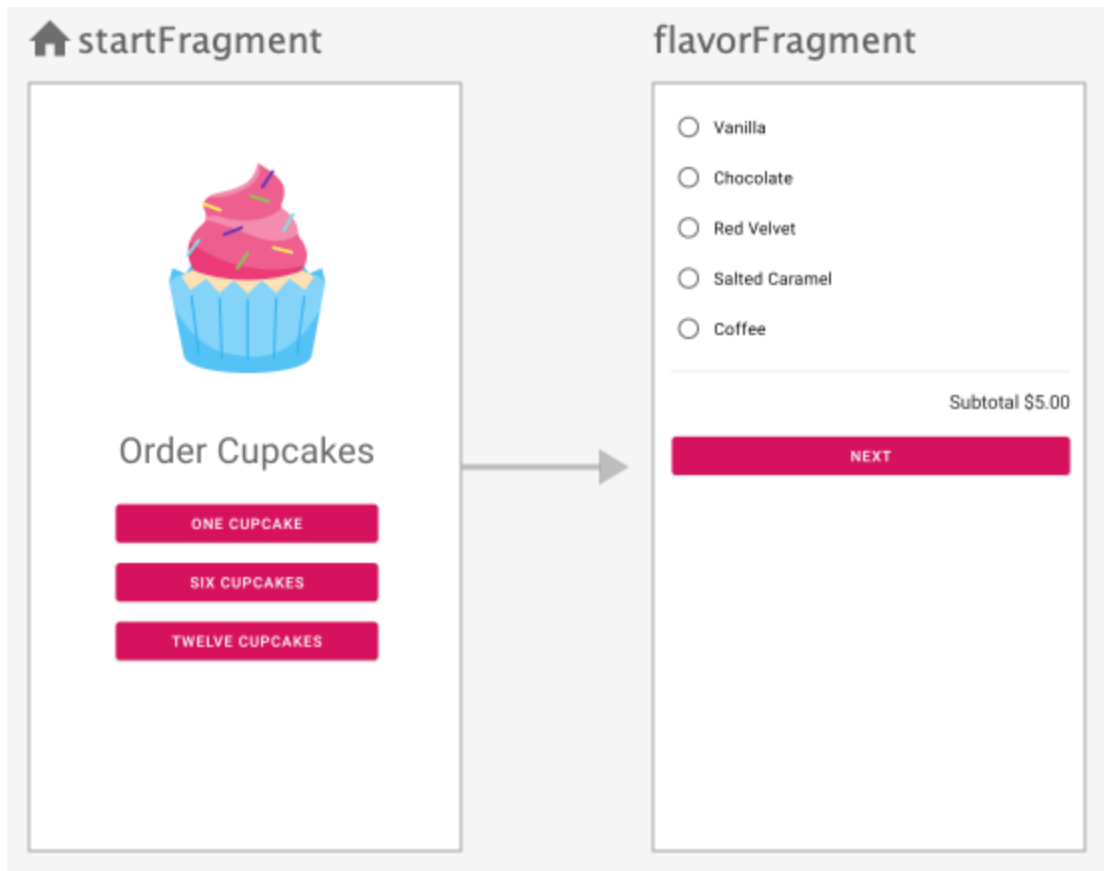


Note: If the destination fragments are laid out differently in your Android Studio, click and drag the destinations to rearrange similarly to the above screenshot. This makes it easier to configure navigation actions later in the codelab.

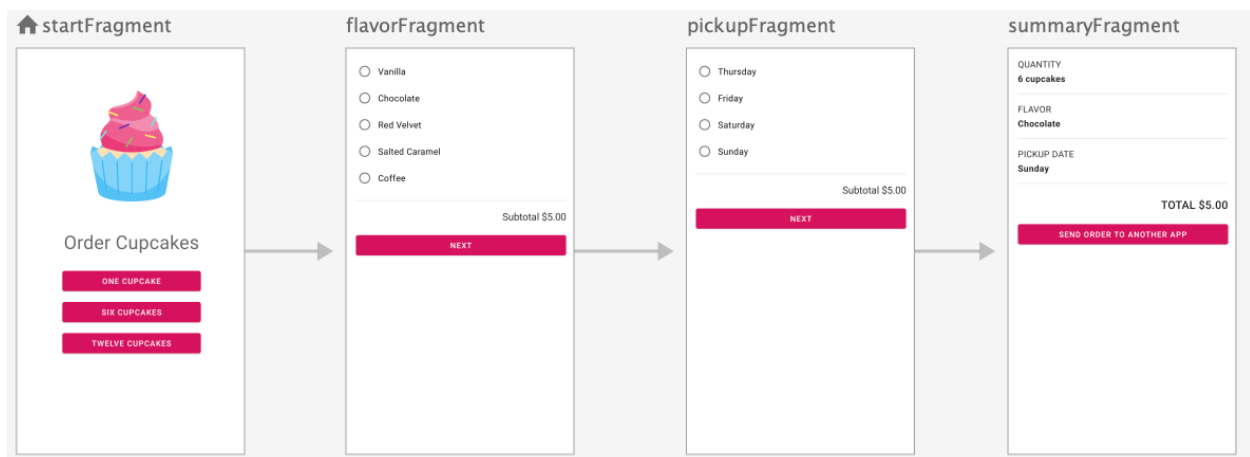
3. Connect the fragment destinations in the nav graph. Create an action from the **startFragment** to the **flavorFragment**, a connection from the **flavorFragment** to the **pickupFragment**, and a connection from the **pickupFragment** to the **summaryFragment**. Follow the next few steps if you need more detailed instructions.
4. Hover over the **startFragment** until you see the gray border around the fragment and the gray circle appear over the center of the right edge of the fragment. Click on the circle and drag to the **flavorFragment**, and then release the mouse.



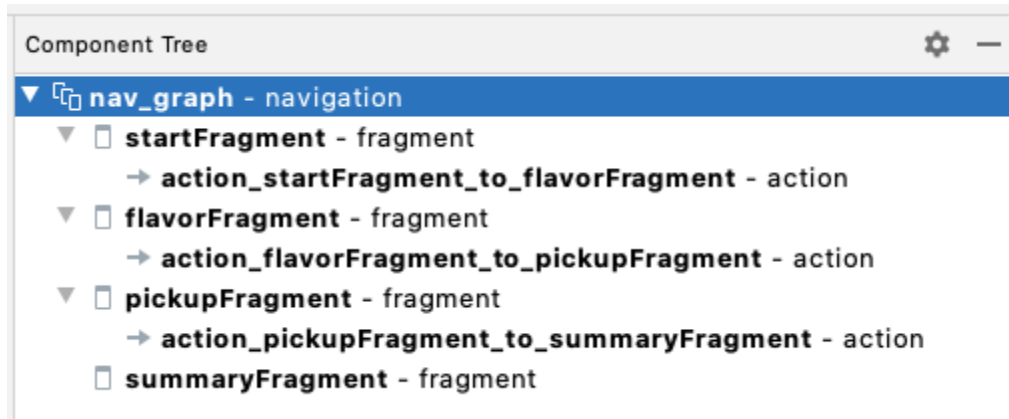
5. An arrow between the two fragments indicates a successful connection, meaning you will be able to navigate from the **startFragment** to the **flavorFragment**. This is called a Navigation action, which you have learned in a previous codelab.



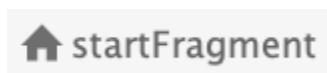
6. Similarly add navigation actions from **flavorFragment** to **pickupFragment** and from **pickupFragment** to **summaryFragment**. When you're done creating the navigation actions, the completed navigation graph should look like the following.



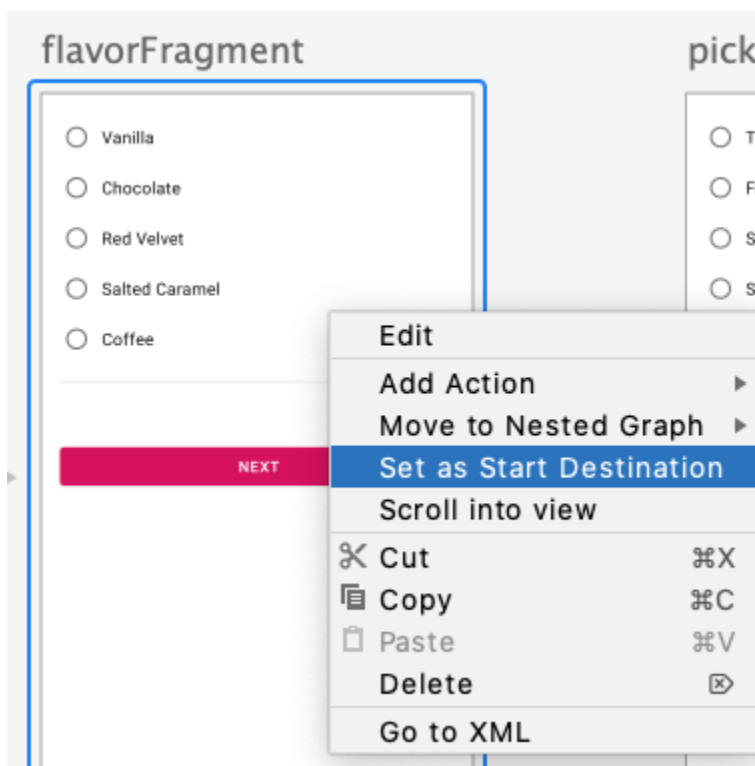
7. The three new actions you created should be reflected in the **Component Tree** pane as well.



- When you define a navigation graph, you also want to specify the start destination. Currently you can see that startFragment has a little house icon next to it.



That indicates that **startFragment** will be the first fragment to be shown in the `NavHost`. Leave this as the desired behavior for our app. For future reference, you can always change the start destination by right clicking on a fragment and selecting the menu option **Set as Start Destination**.



Navigate from start fragment to flavor fragment

Next, you will add code to navigate from **startFragment** to **flavorFragment** by tapping the buttons in the first fragment, instead of displaying a `Toast` message. Below is the reference of the start fragment layout. You will pass the quantity of cupcakes to the flavor fragment in a later task.

11:36



Cupcake



Order Cupcakes

ONE CUPCAKE

SIX CUPCAKES

TWELVE CUPCAKES



1. In the **Project** window, open the **app > java > com.example.cupcake > StartFragment** Kotlin file.
2. In the `onViewCreated()` method, notice the click listeners are set on the three buttons. When each button is tapped, the `orderCupcake()` method is called with the quantity of cupcakes (either 1, 6, or 12 cupcakes) as its parameter.

Reference code:

```
orderOneCupcake.setOnClickListener { orderCupcake(1) }
orderSixCupcakes.setOnClickListener { orderCupcake(6) }
orderTwelveCupcakes.setOnClickListener { orderCupcake(12) }
```

3. In the `orderCupcake()` method, replace the code displaying the toast message with the code to navigate to the flavor fragment. Get the `NavController` using `findNavController()` method and call `navigate()` on it, passing in the action ID, `R.id.action_startFragment_to_flavorFragment`. Make sure this action ID matches the action declared in your `nav_graph.xml`.

Replace

```
fun orderCupcake(quantity: Int) {
    Toast.makeText(activity, "Ordered $quantity cupcake(s)",
        Toast.LENGTH_SHORT).show()
}
```

with

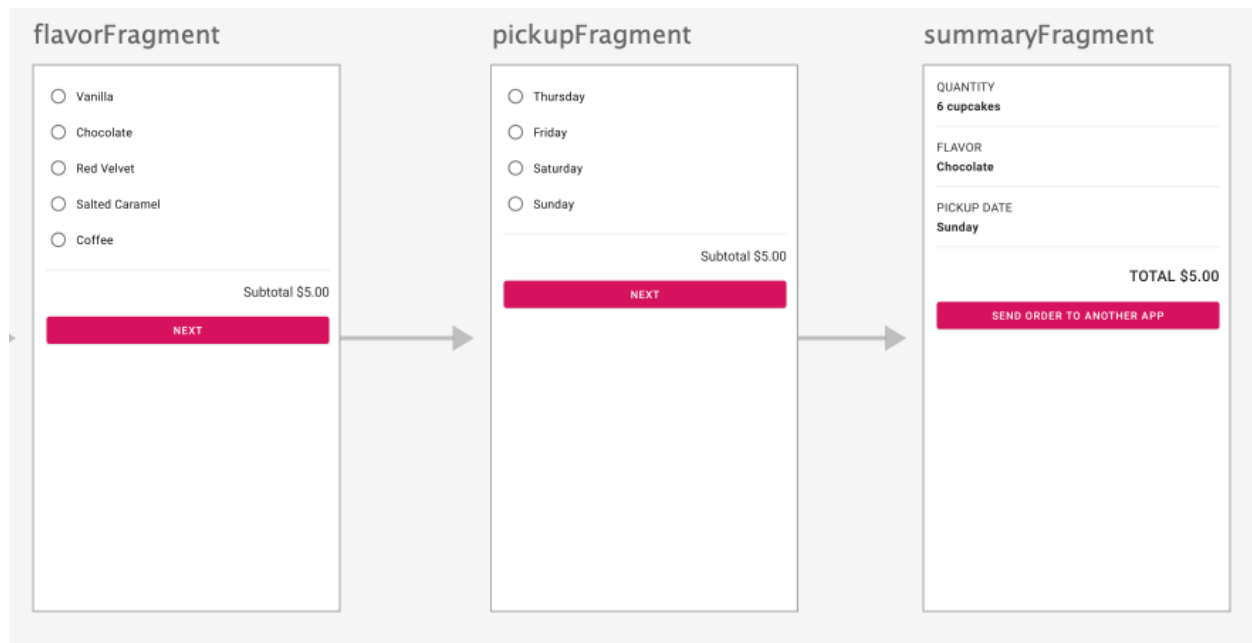
```
fun orderCupcake(quantity: Int) {
    findNavController().navigate(R.id.action_startFragment_to_flavorFragment)
}
```

4. Add the Import `import androidx.navigation.fragment.findNavController` or you can select from the options provided by Android Studio.



Add Navigation to the flavor and pickup fragments

Similar to the previous task, in this task you will add the navigation to the other fragments: flavor and the pickup fragments.



1. Open **app > java > com.example.cupcake > FlavorFragment.kt**. Notice the method called within the **Next** button click listener is `goToNextScreen()` method.
2. In `FlavorFragment.kt`, inside the `goToNextScreen()` method, replace the code displaying the toast to navigate to the pickup fragment. Use the action ID, `R.id.action_flavorFragment_to_pickupFragment` and make sure this ID matches the action declared in your `nav_graph.xml`.

```
fun goToNextScreen() {  
    findNavController().navigate(R.id.action_flavorFragment_to_pickupFragment)  
}  
}
```

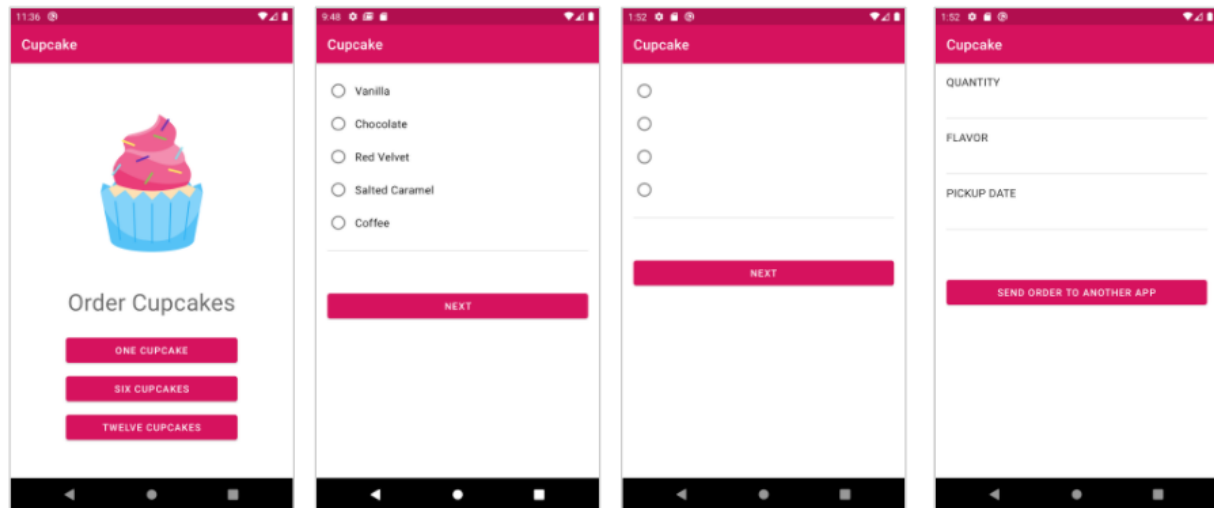
Remember to import `androidx.navigation.fragment.findNavController`.

3. Similarly in `PickupFragment.kt`, inside the `goToNextScreen()` method, replace the existing code to navigate to the summary fragment.

```
fun goToNextScreen() {  
    findNavController().navigate(R.id.action_pickupFragment_to_summaryFragmen  
t)  
}  
}
```

Import `androidx.navigation.fragment.findNavController`.

4. Run the app. Make sure the buttons work to navigate from screen to screen. The information displayed on each fragment may be incomplete, but don't worry, you'll be populating those fragments with the correct data in upcoming steps.



Update title in app bar

As you navigate through the app, notice the title in the app bar. It is always displayed as **Cupcake**.

It would be a better user experience to provide a more relevant title based on the functionality of the current fragment.

Change the title in the app bar (also known as action bar) for each fragment using the `NavController` and display an **Up** (←) button.



1. In `MainActivity.kt`, override the `onCreate()` method to set up the navigation controller. Get an instance of `NavController` from the `NavHostFragment`.
2. Make a call to `setupActionBarWithNavController(navController)` passing in the instance of `NavController`. This will do the following: Show a title in the app bar based off of the destination's label, and display the **Up** button whenever you're not on a top-level destination.

```
class MainActivity : AppCompatActivity(R.layout.activity_main) {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        val navHostFragment = supportFragmentManager
            .findFragmentById(R.id.nav_host_fragment) as NavHostFragment
        val navController = navHostFragment.navController

        setupActionBarWithNavController(navController)
    }
}
```

```
}  
}
```

3. Add necessary imports when prompted by Android Studio.

```
import android.os.Bundle  
import androidx.navigation.fragment.NavHostFragment  
import androidx.navigation.ui.setupActionBarWithNavController
```

4. Set the app bar titles for each fragment. Open `navigation/nav_graph.xml` and switch to **Code** tab.
5. In `nav_graph.xml`, modify the `android:label` attribute for each fragment destination. Use the following string resources that have already been declared in the starter app.

For start fragment, use `@string/app_name` with value `Cupcake`.

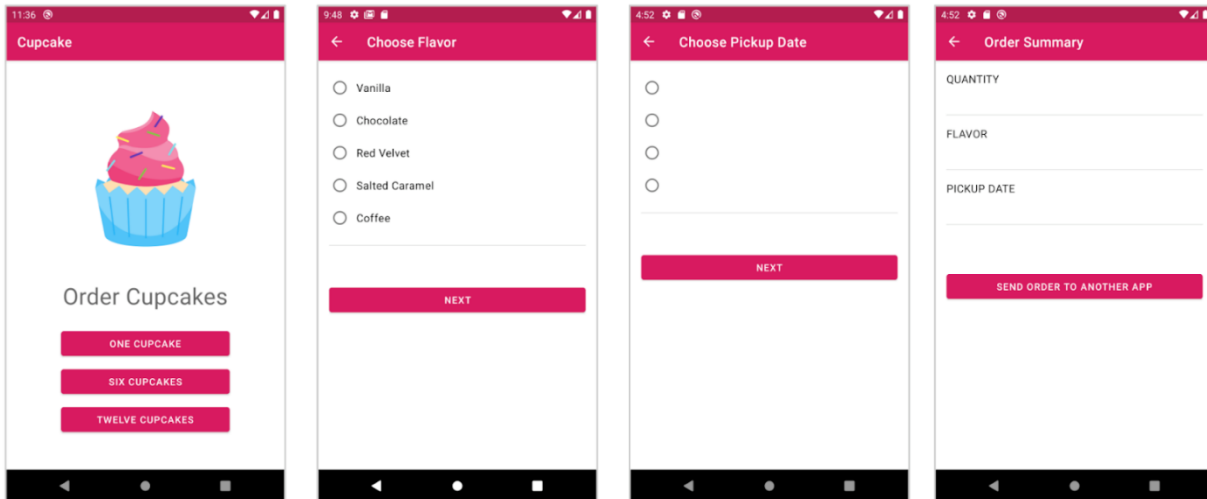
For flavor fragment, use `@string/choose_flavor` with value `Choose Flavor`.

For pickup fragment, use `@string/choose_pickup_date` with value `Choose Pickup Date`.

For summary fragment, use `@string/order_summary` with value `Order Summary`.

```
<navigation ...>  
    <fragment  
        android:id="@+id/startFragment"  
        ...  
        android:label="@string/app_name" ... >  
        <action ... />  
    </fragment>  
    <fragment  
        android:id="@+id/flavorFragment"  
        ...  
        android:label="@string/choose_flavor" ... >  
        <action ... />  
    </fragment>  
    <fragment  
        android:id="@+id/pickupFragment"  
        ...  
        android:label="@string/choose_pickup_date" ... >  
        <action ... />  
    </fragment>  
    <fragment  
        android:id="@+id/summaryFragment"  
        ...  
        android:label="@string/order_summary" ... />  
</navigation>
```

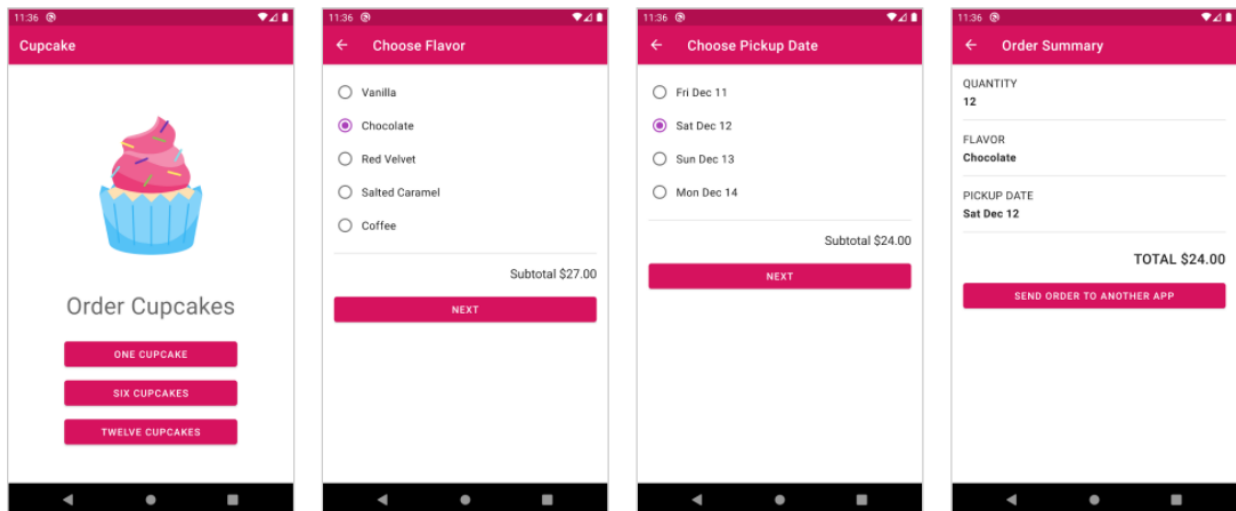
6. Run the app. Notice the title in the app bar changes as you navigate to each fragment destination. Also notice that the **Up** button (arrow ←) is now showing in the app bar. If you tap on it, it doesn't do anything. You will implement the **Up** button behavior in the next codelab.



4. Create a shared ViewModel

Let's move onto populating the correct data in each of the fragments. You'll be using a shared `ViewModel` to save the app's data in a single `ViewModel`. Multiple fragments in the app will access the shared `ViewModel` using their activity scope.

It is a common use case to share data between fragments in most production apps. For example in the final version(of this codelab) of the **Cupcake** app (notice the screenshots below), the user selects the quantity of cupcakes in the first screen, and in the second screen the price is calculated and displayed based on the quantity of the cupcakes. Similarly other app data such as flavor and pickup date are also used in summary screen.



From looking at the app features, you can reason that it would be useful to store this order information in a single `ViewModel`, which can be shared across the fragments in this activity.

Recollect that [ViewModel](#) is a part of the [Android Architecture Components](#). The app data saved within the `ViewModel` is retained during configuration changes. To add a `ViewModel` to your app, you create a new class that extends from the [ViewModel](#) class.

Create OrderViewModel

In this task, you will create a shared `ViewModel` for the **Cupcake** app called `OrderViewModel`. You will also add the app data as properties inside the `ViewModel` and methods to update and modify the data. Here are the properties of the class:

- Order quantity (`Integer`)
- Cupcake flavor (`String`)
- Pickup date (`String`)
- Price (`Double`)

Follow [ViewModel](#) best practices

In a `ViewModel`, it is a recommended practice to *not* expose view model data as `public` variables. Otherwise the app data can be modified in unexpected ways by the external classes and create edge cases your app didn't expect to handle. Instead, make these mutable properties `private`, implement a backing property, and expose a `public` immutable version of each property, if needed. The convention is to prefix the name of the `private` mutable properties with an underscore (`_`).

Here are the methods to update the properties above, depending on the user's choice:

- `setQuantity(numberCupcakes: Int)`
- `setFlavor(desiredFlavor: String)`
- `setDate(pickupDate: String)`

You don't need a setter method for the price because you will calculate it within the `OrderViewModel` using other properties. The steps below walk you through how to implement the shared `ViewModel`.

You will create a new package in your project called `model` and add the `OrderViewModel` class. This will separate out the view model code from the rest of your UI code (fragments and activities). It is a coding best practice to separate code into packages depending on the functionality.

1. In the **Project** window of Android Studio, right click on **com.example.cupcake** > **New** > **Package**.
2. A **New Package** dialog will be opened, give the package name as `com.example.cupcake.model`.

New Package

`com.example.cupcake.model`

3. Create the `OrderViewModel` Kotlin class under the `model` package. In the **Project** window, right-click on the `model` package and select **New > Kotlin File/Class**. In the new dialog, give the filename `OrderViewModel`.

New Kotlin File/Class

 `OrderViewModel`

 File

 **Class**

 Interface

 Enum class

 Object

4. In `OrderViewModel.kt`, change the class signature to extend from `ViewModel`.

```
import androidx.lifecycle.ViewModel

class OrderViewModel : ViewModel() {

}
```

5. Inside the `OrderViewModel` class, add the properties that were discussed above as `private val`.
6. Change the property types to `LiveData` and add backing fields to the properties, so that these properties can be observable and UI can be updated when the source data in the view model changes.

```
private val _quantity = MutableLiveData<Int>(0)
val quantity: LiveData<Int> = _quantity

private val _flavor = MutableLiveData<String>("")
val flavor: LiveData<String> = _flavor
```

```
private val _date = MutableLiveData<String>("")
val date: LiveData<String> = _date

private val _price = MutableLiveData<Double>(0.0)
val price: LiveData<Double> = _price
```

You will need to import these classes:

```
import androidx.lifecycle.LiveData
import androidx.lifecycle.MutableLiveData
```

7. In `OrderViewModel` class, add the methods that were discussed above. Inside the methods, assign the argument passed in to the mutable properties.
8. Since these setter methods need to be called from outside the view model, leave them as public methods (meaning no `private` or other visibility modifier needed before the `fun` keyword). The default visibility modifier in Kotlin is `public`.

```
fun setQuantity(numberCupcakes: Int) {
    _quantity.value = numberCupcakes
}

fun setFlavor(desiredFlavor: String) {
    _flavor.value = desiredFlavor
}

fun setDate(pickupDate: String) {
    _date.value = pickupDate
}
```

9. Build and run your app to make sure there are no compile errors. There should be no visible change in your UI yet.

Nice work! Now you have the start to your view model. You'll incrementally add more to this class as you build out more features in your app and realize you need more properties and methods in your class.

If you see the class names, property names, or method names in gray font in Android Studio, that's expected. That means the class, properties, or methods are not being used at the moment, but they will be! That's coming up next.

5. Use the ViewModel to update the UI

In this task, you will use the shared view model you created to update the app's UI. The main difference in the implementation of a shared view model is the way we access it from the UI controllers. You will use the activity instance instead of the fragment instance, and you will see how to do this in the coming sections.

That means the view model can be shared across fragments. Each fragment could access the view model to check on some detail of the order or update some data in the view model.

Update StartFragment to use view model

To use the shared view model in `StartFragment` you will initialize the `OrderViewModel` using `activityViewModels()` instead of `viewModels()` delegate class.

- `viewModels()` gives you the `ViewModel` instance scoped to the current fragment. This will be different for different fragments.
- `activityViewModels()` gives you the `ViewModel` instance scoped to the current activity. Therefore the instance will remain the same across multiple fragments in the same activity.

Use Kotlin property delegate

In Kotlin, each mutable (`var`) property has default getter and setter functions automatically generated for it. The setter and getter functions are called when you assign a value or read the value of the property. (For a read-only property (`val`), only the getter function is generated by default. This getter function is called when you read the value of a read-only property.)

Property delegation in Kotlin helps you to handoff the getter-setter responsibility to a different class.

This class (called *delegate class*) provides getter and setter functions of the property and handles its changes.

A delegate property is defined using the `by` clause and a delegate class instance:

```
// Syntax for property delegation
var <property-name> : <property-type> by <delegate-class>()
```

1. In `StartFragment` class, get a reference to the shared view model as a class variable. Use the `by activityViewModels()` Kotlin property delegate from the `fragment-ktx` library.

```
private val sharedViewModel: OrderViewModel by activityViewModels()
```

You may need these new imports:

```
import androidx.fragment.app.activityViewModels
import com.example.cupcake.model.OrderViewModel
```

2. Repeat the above step for `FlavorFragment`, `PickupFragment`, `SummaryFragment` classes, you will use this `sharedViewModel` instance in later sections of the codelab.

3. Going back to the `StartFragment` class, you can now use the view model. At the beginning of the `orderCupcake()` method, call the `setQuantity()` method in the shared view model to update quantity, before navigating to the flavor fragment.

```
fun orderCupcake(quantity: Int) {  
    sharedViewModel.setQuantity(quantity)  
    findNavController().navigate(R.id.action_startFragment_to_flavorFragment)  
}
```

4. Within the `OrderViewModel` class, add the following method to check if the flavor for the order has been set or not. You will use this method in the `StartFragment` class in a later step.

```
fun hasNoFlavorSet(): Boolean {  
    return _flavor.value.isNullOrEmpty()  
}
```

5. In `StartFragment` class, inside `orderCupcake()` method, after setting the quantity, set the default flavor as Vanilla if no flavor is set, before navigating to the flavor fragment. Your complete method will look like this:

```
fun orderCupcake(quantity: Int) {  
    sharedViewModel.setQuantity(quantity)  
    if (sharedViewModel.hasNoFlavorSet()) {  
        sharedViewModel.setFlavor(getString(R.string.vanilla))  
    }  
    findNavController().navigate(R.id.action_startFragment_to_flavorFragment)  
}
```

6. Build the app to make sure there are no compile errors. There should be no visible change in your UI though.

6. Use ViewModel with data binding

Next you will use data binding to bind the view model data to the UI. You will also update the shared view model based on the selections the user makes in the UI.

Refresher on Data binding

Recall that the [Data Binding Library](#) is a part of [Android Jetpack](#). Data binding binds the UI components in your layouts to data sources in your app using a declarative format. In simpler terms, data binding is binding data (from code) to views + view binding (binding views to code). By setting up these bindings and having updates be automatic, this helps you reduce the chance for errors if you forget to manually update the UI from your code.

Update flavor with user choice

1. In `layout/fragment_flavor.xml`, add a `<data>` tag inside the root `<layout>` tag. Add a layout variable called `viewModel` of the type `com.example.cupcake.model.OrderViewModel`. Make sure the package name in the `type` attribute matches with the package name of the shared view model class, `OrderViewModel` in your app.

```
<layout ...>

    <data>
        <variable
            name="viewModel"
            type="com.example.cupcake.model.OrderViewModel" />
    </data>

    <ScrollView ...>

        ...
```

2. Similarly, repeat the above step for `fragment_pickup.xml`, and `fragment_summary.xml` to add the `viewModel` layout variable. You will use this variable in later sections. You don't need to add this code in `fragment_start.xml`, because this layout doesn't use the shared view model.
3. In the `FlavorFragment` class, inside `onViewCreated()`, bind the view model instance with the shared view model instance in the layout. Add the following code inside the `binding?.apply` block.

```
binding?.apply {
    viewModel = sharedViewModel
    ...
}
```

Apply scope function

This may be the first time you're seeing the `apply` function in Kotlin. `apply` is a [scope function](#) in the Kotlin standard library. It executes a block of code within the context of an object. It forms a temporary scope, and in that scope, you can access the object without its name. The common use case for `apply` is to configure an object. Such calls can be read as *"apply the following assignments to the object."*

Example:

```
clark.apply {
    firstName = "Clark"
    lastName = "James"
    age = 18
}
```

// The equivalent code without `apply` scope function would look like the following.

```
clark.firstName = "Clark"
clark.lastName = "James"
clark.age = 18
```

4. Repeat the same step for the `onViewCreated()` method inside the `PickupFragment` and `SummaryFragment` classes.

```
binding?.apply {
    viewModel = sharedViewModel
    ...
}
```

5. In `fragment_flavor.xml`, use the new layout variable, `viewModel` to set the `checked` attribute of the radio buttons based on the `flavor` value in the view model. If the flavor represented by a radio button is the same as the flavor that's saved in the view model, then display the radio button as selected (`checked = true`). The binding expression for the checked state of the **Vanilla** `RadioButton` would look like the following:

```
@{viewModel.flavor.equals(@string/vanilla)}
```

Essentially, you are comparing the `viewModel.flavor` property with the corresponding string resource using the [equals](#) function, to determine if the checked state should be true or false.

Note: Remember that binding expressions start with an `@` symbol and are wrapped inside curly braces `{}`.

```
<RadioGroup
    ...>

    <RadioButton
        android:id="@+id/vanilla"
        ...
        android:checked="@{viewModel.flavor.equals(@string/vanilla)}"
        .../>

    <RadioButton
        android:id="@+id/chocolate"
        ...
        android:checked="@{viewModel.flavor.equals(@string/chocolate)}"
        .../>

    <RadioButton
        android:id="@+id/red_velvet"
        ...
        android:checked="@{viewModel.flavor.equals(@string/red_velvet)}"
        .../>

    <RadioButton
        android:id="@+id/salted_caramel"
        ...
        android:checked="@{viewModel.flavor.equals(@string/salted_caramel)}"
        .../>
```

```

<RadioButton
    android:id="@+id/coffee"
    ...
    android:checked="@{viewModel.flavor.equals(@string/coffee)}"
    .../>
</RadioGroup>

```

Listener bindings

Listener bindings are lambda expressions that run when an event happens, such as an `onClick` event. They are similar to method references such as `textView.setOnClickListener(clickListener)` but listener bindings let you run arbitrary data binding expressions.

1. In `fragment_flavor.xml`, add event listeners to the radio buttons using listener bindings. Use a lambda expression with no parameters and make a call to the `viewModel.setFlavor()` method by passing in the corresponding flavor string resource.

```

<RadioGroup
    ...>

    <RadioButton
        android:id="@+id/vanilla"
        ...
        android:onClick="@{() -> viewModel.setFlavor(@string/vanilla)}"
        .../>

    <RadioButton
        android:id="@+id/chocolate"
        ...
        android:onClick="@{() -> viewModel.setFlavor(@string/chocolate)}"
        .../>

    <RadioButton
        android:id="@+id/red_velvet"
        ...
        android:onClick="@{() -> viewModel.setFlavor(@string/red_velvet)}"
        .../>

    <RadioButton
        android:id="@+id/salted_caramel"
        ...
        android:onClick="@{() -> viewModel.setFlavor(@string/salted_caramel)}"
        .../>

    <RadioButton
        android:id="@+id/coffee"
        ...
        android:onClick="@{() -> viewModel.setFlavor(@string/coffee)}"
        .../>
</RadioGroup>

```

7. Run the app and notice how the **Vanilla** option is selected by default in the flavor fragment.

12:42



Choose Flavor



Vanilla



Chocolate



Red Velvet



Salted Caramel



Coffee

NEXT



Great! Now you can move onto the next fragments.

7. Update pickup and summary fragment to use view model

Navigate through the app and notice that in the pickup fragment, the radio button option labels are blank. In this task, you will calculate the 4 pickup dates available and display them in the pickup fragment. There are different ways to display a formatted date, and here are some helpful utilities provided by Android to do this.

Create pickup options list

Date formatter

The Android framework provides a class called [SimpleDateFormat](#), which is a class for formatting and parsing dates in a locale-sensitive manner. It allows for formatting (date → text) and parsing (text → date) of dates.

You can create an instance of `SimpleDateFormat` by passing in a pattern string and a locale:

```
SimpleDateFormat("E MMM d", Locale.getDefault())
```

A pattern string like "E MMM d" is a representation of Date and Time formats. Letters from 'A' to 'Z' and from 'a' to 'z' are interpreted as pattern letters representing the components of a date or time string. For example, `d` represents day in a month, `y` for year and `M` for month. If the date is January 4 in 2018, the pattern string "EEE, MMM d" parses to "Wed, Jul 4". For a complete list of pattern letters, please see the [documentation](#).

A [Locale](#) object represents a specific geographical, political, or cultural region. It represents a language/country/variant combination. Locales are used to alter the presentation of information such as numbers or dates to suit the conventions in the region. Date and time are locale-sensitive, because they are written differently in different parts of the world. You will use the method `Locale.getDefault()` to retrieve the locale information set on the user's device and pass it into the `SimpleDateFormat` constructor.

Locale in Android is a combination of language and country code. The language codes are two-letter lowercase ISO language codes, such as "en" for english. The country codes are two-letter uppercase ISO country codes, such as "US" for the United States.

Now use `SimpleDateFormat` and `Locale` to determine the available pickup dates for the **Cupcake** app.

1. In `OrderViewModel` class, add the following function called `getPickupOptions()` to create and return the list of pickup dates. Within the method, create a `val` variable called `options` and initialize it to `mutableListOf<String>()`.

```
private fun getPickupOptions(): List<String> {
    val options = mutableListOf<String>()
}
```

2. Create a formatter string using `SimpleDateFormat` passing pattern string "E MMM d", and the locale. In the pattern string, E stands for day name in week and it parses to "**Tue Dec 10**".

```
val formatter = SimpleDateFormat("E MMM d", Locale.getDefault())
```

Import `java.text.SimpleDateFormat` and `java.util.Locale`, when prompted by Android Studio.

3. Get a `Calendar` instance and assign it to a new variable. Make it a `val`. This variable will contain the current date and time. Also, import `java.util.Calendar`.

```
val calendar = Calendar.getInstance()
```

4. Build up a list of dates starting with the current date and the following three dates. Because you'll need 4 date options, repeat this block of code 4 times. This [repeat](#) block will format a date, add it to the list of date options, and then increment the calendar by 1 day.

```
repeat(4) {
    options.add(formatter.format(calendar.time))
    calendar.add(Calendar.DATE, 1)
}
```

5. Return the updated `options` at the end of the method. Here is your completed method:

```
private fun getPickupOptions(): List<String> {
    val options = mutableListOf<String>()
    val formatter = SimpleDateFormat("E MMM d", Locale.getDefault())
    val calendar = Calendar.getInstance()
    // Create a list of dates starting with the current date and the following
3 dates
    repeat(4) {
        options.add(formatter.format(calendar.time))
        calendar.add(Calendar.DATE, 1)
    }
    return options
}
```

6. In `OrderViewModel` class, add a class property called `dateOptions` that's a `val`. Initialize it using the `getPickupOptions()` method you just created.

```
val dateOptions = getPickupOptions()
```

Update the layout to display pickup options

Now that you have the four available pickup dates in the view model, update the `fragment_pickup.xml` layout to display these dates. You will also use data binding to display the checked status of each radio button and to update the date in the view model when a different radio button is selected. This implementation is similar to the data binding in the flavor fragment.

In `fragment_pickup.xml`:

Radio button `option0` represents `dateOptions[0]` in `viewModel` (today)

Radio button `option1` represents `dateOptions[1]` in `viewModel` (tomorrow)

Radio button `option2` represents `dateOptions[2]` in `viewModel` (the day after tomorrow)

Radio button `option3` represents `dateOptions[3]` in `viewModel` (two days after tomorrow)

1. In `fragment_pickup.xml`, for the `option0` radio button, use the new layout variable, `viewModel` to set the `checked` attribute based on the `date` value in the view model. Compare the `viewModel.date` property with the first string in the `dateOptions` list, which is the current date. Use the [equals](#) function to compare and the final binding expression looks like the following:

```
@{viewModel.date.equals(viewModel.dateOptions[0])}
```

2. For the same radio button, add an event listener using listener binding to the `onClick` attribute. When this radio button option is clicked, make a call to `setDate()` on `viewModel`, passing in `dateOptions[0]`.
3. For the same radio button, set the `text` attribute value to the first string in the `dateOptions` list.

```
<RadioButton
    android:id="@+id/option0"
    ...
    android:checked="@{viewModel.date.equals(viewModel.dateOptions[0])}"
    android:onClick="@{() -> viewModel.setDate(viewModel.dateOptions[0])}"
    android:text="@{viewModel.dateOptions[0]}"
    ...
/>
```

4. Repeat the above steps for the other radio buttons, change the index of the `dateOptions` accordingly.

```
<RadioButton
    android:id="@+id/option1"
    ...
    android:checked="@{viewModel.date.equals(viewModel.dateOptions[1])}"
    android:onClick="@{() -> viewModel.setDate(viewModel.dateOptions[1])}"
    android:text="@{viewModel.dateOptions[1]}"
    ... />
```

```

<RadioButton
    android:id="@+id/option2"
    ...
    android:checked="@{viewModel.date.equals(viewModel.dateOptions[2])}"
    android:onClick="@{() -> viewModel.setDate(viewModel.dateOptions[2])}"
    android:text="@{viewModel.dateOptions[2]}"
    ... />

<RadioButton
    android:id="@+id/option3"
    ...
    android:checked="@{viewModel.date.equals(viewModel.dateOptions[3])}"
    android:onClick="@{() -> viewModel.setDate(viewModel.dateOptions[3])}"
    android:text="@{viewModel.dateOptions[3]}"
    ... />

```

5. Run the app and you should see the next few days as pickup options available. Your screenshot will differ depending on what the current day is for you. Notice that there is no option selected by default. You will implement this in the next step.



Choose Pickup Date

- ☐ Fri Nov 20
 - ☐ Sat Nov 21
 - ☐ Sun Nov 22
 - ☐ Mon Nov 23
-

NEXT



6. Within the `OrderViewModel` class, create a function called `resetOrder()`, to reset the `MutableLiveData` properties in the view model. Assign the current date value from the `dateOptions` list to `_date.value`.

```
fun resetOrder() {  
    _quantity.value = 0  
    _flavor.value = ""  
    _date.value = dateOptions[0]  
    _price.value = 0.0  
}
```

7. Add an `init` block to the class, and call the new method `resetOrder()` from it.

```
init {  
    resetOrder()  
}
```

8. Remove the initial values from the declaration of the properties in the class. Now you are using the `init` block to initialize the properties when an instance of `OrderViewModel` is created.

```
private val _quantity = MutableLiveData<Int>()  
val quantity: LiveData<Int> = _quantity  
  
private val _flavor = MutableLiveData<String>()  
val flavor: LiveData<String> = _flavor  
  
private val _date = MutableLiveData<String>()  
val date: LiveData<String> = _date  
  
private val _price = MutableLiveData<Double>()  
val price: LiveData<Double> = _price
```

9. Run your app again, notice today's date is selected by default.



Choose Pickup Date

☒ Thu Dec 10

☐ Fri Dec 11

☐ Sat Dec 12

☐ Sun Dec 13

NEXT

Update Summary fragment to use view model

Now let's move onto the last fragment. The order summary fragment is intended to show a summary of the order details. In this task, you take advantage of all the order information from the shared view model and update the onscreen order details using data binding.

4:52   



Order Summary

QUANTITY

FLAVOR

PICKUP DATE

SEND ORDER TO ANOTHER APP



1. In `fragment_summary.xml`, make sure you have the view model data variable, `viewModel` **declared**.

```
<layout ...>

    <data>
        <variable
            name="viewModel"
            type="com.example.cupcake.model.OrderViewModel" />
    </data>

    <ScrollView ...>

        ...
```

2. In `SummaryFragment`, in `onViewCreated()`, make sure `binding.viewModel` is initialized.
3. In `fragment_summary.xml`, read from the view model to update the screen with the order summary details. Update the quantity, flavor, and date `TextViews` by adding the following text attributes. Quantity is of the type `Int`, so you need to convert it to a string.

```
<TextView
    android:id="@+id/quantity"
    ...
    android:text="@{viewModel.quantity.toString()}"
    ... />
<TextView
    android:id="@+id/flavor"
    ...
    android:text="@{viewModel.flavor}"
    ... />
<TextView
    android:id="@+id/date"
    ...
    android:text="@{viewModel.date}"
    ... />
```

4. Run and test the app to verify that the order options you selected show up in the order summary.



Order Summary

QUANTITY

1

FLAVOR

Salted Caramel

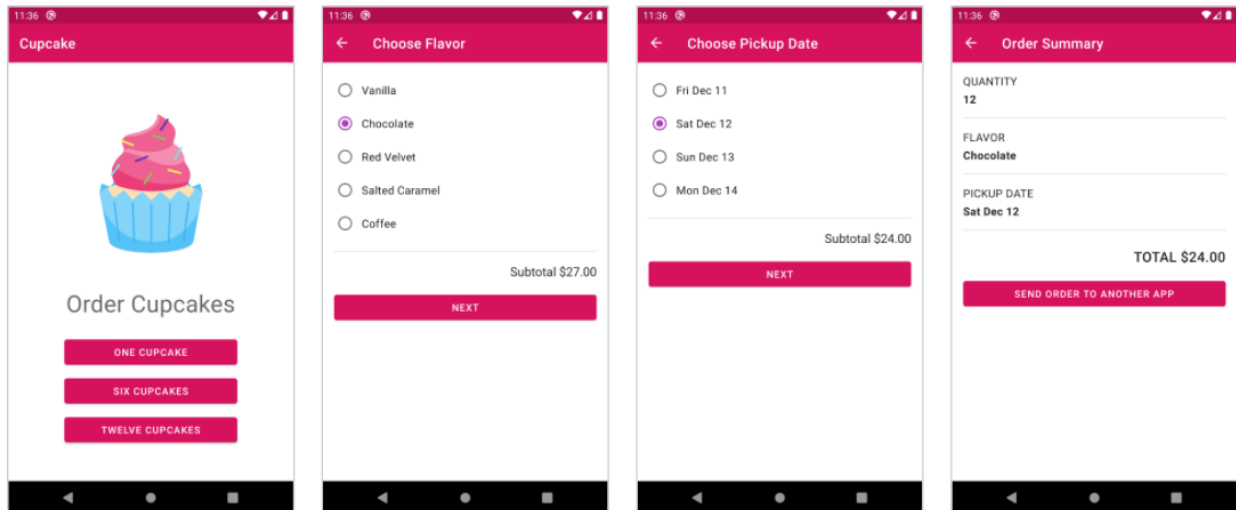
PICKUP DATE

Sat Dec 12

SEND ORDER TO ANOTHER APP

8. Calculate price from order details

Looking at the final app screenshots of this codelab, you'll notice that the price is actually displayed on each fragment (except the `StartFragment`) so the user knows the price as they create the order.



Here are the rules from our cupcake shop on how to calculate price.

- Each cupcake is \$2.00 each
- Same day pickup adds an extra \$3.00 to the order

Hence, for an order of 6 cupcakes, the price would be 6 cupcakes x \$2 each = \$12. If the user wants same day pickup, the extra \$3 cost would lead to a total order price of \$15.

Update price in view model

To add support for this functionality in your app, first tackle the price per cupcake and ignore the same day pickup cost for now.

1. Open `OrderViewModel.kt`, and store the price per cupcake in a variable. Declare it as a top-level private constant at the top of the file, outside the class definition (but after the import statements). Use the `const` modifier and to make it read-only use `val`.

```
package ...

import ...

private const val PRICE_PER_CUPCAKE = 2.00

class OrderViewModel : ViewModel() {
    ...
}
```

Recollect that constant values (marked with the `const` keyword in Kotlin) do not change and the value is known at compile time. To learn more about constants, check out the [documentation](#).

2. Now that you have defined a price per cupcake, create a helper method to calculate the price. This method can be `private` because it's only used within this class. You will change the price logic to include same day pickup charges in the next task.

```
private fun updatePrice() {  
    _price.value = (quantity.value ?: 0) * PRICE_PER_CUPCAKE  
}
```

This line of code multiplies the price per cupcake by the quantity of cupcakes ordered. For the code in parentheses, since the value of `quantity.value` could be null, use an elvis operator (`?:`). The elvis operator (`?:`) means that if the expression on the left is not null, then use it. Otherwise if the expression on the left is null, then use the expression to the right of the elvis operator (which is `0` in this case).

Fun fact: Elvis operator (`?:`) is named after the rock star, Elvis Presley, because when you view it sideways, it resembles an emoticon of [Elvis Presley](#) with his [quiff](#).

3. In the same `OrderViewModel` class, update the price variable when the quantity is set. Make a call to the new function in the `setQuantity()` function.

```
fun setQuantity(numberCupcakes: Int) {  
    _quantity.value = numberCupcakes  
    updatePrice()  
}
```

Bind the price property to the UI

1. In the layouts for `fragment_flavor.xml`, `fragment_pickup.xml`, and `fragment_summary.xml`, make sure the data variable `viewModel` of type `com.example.cupcake.model.OrderViewModel` is defined.

```
<layout ...>  
  
    <data>  
        <variable  
            name="viewModel"  
            type="com.example.cupcake.model.OrderViewModel" />  
    </data>  
  
    <ScrollView ...>  
  
        ...  
    </ScrollView>  
</layout>
```

2. In the `onViewCreated()` method of each fragment class, make sure you bind the view model object instance in the fragment to the view model data variable in the layout.

```
binding?.apply {
    viewModel = sharedViewModel
    ...
}
```

3. Within each fragment layout, use the `viewModel` variable to set the price if it's shown in the layout. Start with modifying the `fragment_flavor.xml` file. For the `subtotal` text view, set the value of the `android:text` attribute to be `"#{@string/subtotal_price(viewModel.price)}"`. This data binding layout expression uses the string resource `@string/subtotal_price` and passes in a parameter, which is the price from the view model, so the output will show **Subtotal 12.0** for example.

```
...
<TextView
    android:id="@+id/subtotal"
    android:text="@{@string/subtotal_price(viewModel.price)}"
    ... />
...
```

You're using this string resource that was already declared in the `strings.xml` file:

```
<string name="subtotal_price">Subtotal %s</string>
```

4. Run the app. If you select **One cupcake** in the start fragment, the flavor fragment will show **Subtotal 2.0**. If you select **Six cupcakes**, the flavor fragment will show **Subtotal 12.0**, and etc... You will format the price into the proper currency format later, so this behavior is expected for now.



Choose Flavor



Vanilla



Chocolate



Red Velvet



Salted Caramel



Coffee

NEXT



Choose Flavor



Vanilla



Chocolate



Red Velvet



Salted Caramel



Coffee

NEXT

5. Now make a similar change for the pickup and summary fragments. In `fragment_pickup.xml` and `fragment_summary.xml` layouts, modify the text views to use the `viewModel.price` property as well.

`fragment_pickup.xml`

...

```
<TextView
    android:id="@+id/subtotal"
    ...
    android:text="@{@string/subtotal_price(viewModel.price)}"
    ... />
```

...

`fragment_summary.xml`

...

```
<TextView
    android:id="@+id/total"
    ...
    android:text="@{@string/total_price(viewModel.price)}"
    ... />
```

...

4. Run the app. Make sure the price shown in the order summary is calculated correctly for an order quantity of 1, 6, and 12 cupcakes. As mentioned, it's expected that the price formatting isn't correct at the moment (it'll show up as 2.0 for \$2 or 12.0 for \$12).



Choose Pickup Date

☒ Thu Dec 10

☐ Fri Dec 11

☐ Sat Dec 12

☐ Sun Dec 13

NEXT



Order Summary

QUANTITY

6

FLAVOR

Vanilla

PICKUP DATE

Thu Dec 10

SEND ORDER TO ANOTHER ADDRESS

Charge extra for same day pickup

In this task, you will implement the second rule which is that same day pickup adds an extra \$3.00 to the order.

1. In `OrderViewModel` class, define a new top-level private constant for the same day pickup cost.

```
private const val PRICE_FOR_SAME_DAY_PICKUP = 3.00
```

2. In `updatePrice()`, check if the user selected the same day pickup. Check if the date in the view model (`_date.value`) is the same as the first item in the `dateOptions` list which is always the current day.

```
private fun updatePrice() {  
    _price.value = (quantity.value ?: 0) * PRICE_PER_CUPCAKE  
    if (dateOptions[0] == _date.value) {  
  
    }  
}
```

3. To make these calculations simpler, introduce a temporary variable, `calculatedPrice`. Calculate the updated price and assign it back to `_price.value`.

```
private fun updatePrice() {  
    var calculatedPrice = (quantity.value ?: 0) * PRICE_PER_CUPCAKE  
    // If the user selected the first option (today) for pickup, add the  
    surcharge  
    if (dateOptions[0] == _date.value) {  
        calculatedPrice += PRICE_FOR_SAME_DAY_PICKUP  
    }  
    _price.value = calculatedPrice  
}
```

4. Call `updatePrice()` helper method from `setDate()` method to add the same day pickup charges.

```
fun setDate(pickupDate: String) {  
    _date.value = pickupDate  
    updatePrice()  
}
```

5. Run your app, navigate through the app. You will notice that changing the pickup date does not remove the same day pickup charges from the total price. This is because the price is changed in the view model but it is not notified to the binding layout.



Choose Pickup Date

☐ Wed Dec 16

☒ Thu Dec 17

☐ Fri Dec 18

☐ Sat Dec 19

Subtotal 5.0

NEXT

Set Lifecycle owner to observe LiveData

[LifecycleOwner](#) is a class that has an Android lifecycle, such as an activity or a fragment. A `LiveData` observer observes the changes to the app's data only if the lifecycle owner is in active states (`STARTED` or `RESUMED`).

In your app, the `LiveData` object or the observable data is the `price` property in the view model. The lifecycle owners are the flavor, pickup and the summary fragments. The `LiveData` observers are the binding expressions in layout files with observable data like price. With Data Binding, when an observable value changes, the UI elements it's bound to are updated automatically.

Example of binding expression:

```
android:text="@{@string/subtotal_price(viewModel.price)}"
```

For the UI elements to automatically update, you have to associate `binding.lifecycleOwner` with the lifecycle owners in the app. You will implement this next.

1. In the `FlavorFragment`, `PickupFragment`, `SummaryFragment` classes, inside the `onViewCreated()` method, add the following in the `binding?.apply` block. This will set the lifecycle owner on the binding object. By setting the lifecycle owner, the app will be able to observe `LiveData` objects.

```
binding?.apply {  
    lifecycleOwner = viewLifecycleOwner  
    ...  
}
```

2. Run your app again. In the pickup screen, change the pickup date and notice the difference in how the price changes automatically. And the pick up charges are correctly reflected in the summary screen.
3. Notice that when you select today's date for pickup, the price of the order is increased by \$3.00. The price for selecting any future date should still be the quantity of cupcakes x \$2.00.



Choose Pickup Date

☒ Wed Dec 16

☐ Thu Dec 17

☐ Fri Dec 18

☐ Sat Dec 19

NEXT



Choose Pickup Date

☐ Wed Dec 16

☒ Thu Dec 17

☐ Fri Dec 18

☐ Sat Dec 19

NEXT



4. Test different cases with different cupcake quantities, flavors, and pickup dates. Now you should see the price updating from the view model on each fragment. The best part is that you didn't have to write extra Kotlin code to keep the UI updated with the price each time.

10:50   



Order Summary

QUANTITY

6

FLAVOR

Red Velvet

PICKUP DATE

Sun Dec 13

TOTAL 12.0

SEND ORDER TO ANOTHER APP



To finish implementing the price feature, you'll need to format the price to the local currency.

Format price with LiveData transformation

The `LiveData` transformation method(s) provides a way to perform data manipulations on the source `LiveData` and return a resulting `LiveData` object. In simple terms, it transforms the value of `LiveData` into another value. These transformations aren't calculated unless an observer is observing the `LiveData` object.

The [`Transformations.map\(\)`](#) is one of the transformation functions, this method takes the source `LiveData` and a function as parameters. The function manipulates the source `LiveData` and returns an updated value which is also observable.

Some real-time examples where you may use a `LiveData` transformation:

- Format date, time strings for display
- Sorting a list of items
- Filtering or grouping the items
- Calculate the result from a list like sum of all the items, number of items, return the last item, and so on.

In this task, you will use [`Transformations.map\(\)`](#) method to format the price to use the local currency. You'll transform the original price as a decimal value (`LiveData<Double>`) into a string value (`LiveData<String>`).

1. In `OrderViewModel` class, change the backing property type to `LiveData<String>` instead of `LiveData<Double>`. The formatted price will be a string with a currency symbol such as a '\$'. You will fix the initialization error in the next step.

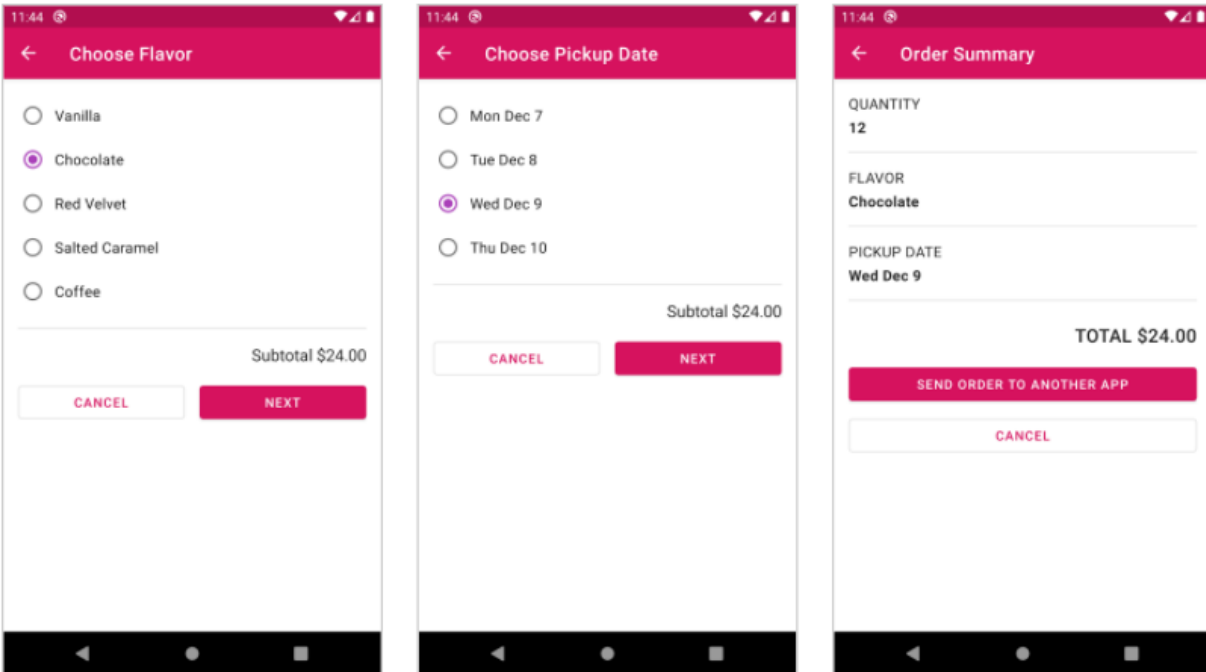
```
private val _price = MutableLiveData<Double>()
val price: LiveData<String>
```

2. Use [`Transformations.map\(\)`](#) to initialize the new variable, pass in the `_price` and a lambda function. Use `getCurrencyInstance()` method in the [`NumberFormat`](#) class to convert the price to local currency format. The transformation code will look like this.

```
private val _price = MutableLiveData<Double>()
val price: LiveData<String> = Transformations.map(_price) {
    NumberFormat.getCurrencyInstance().format(it)
}
```

You'll need to import `androidx.lifecycle.Transformations` and `java.text.NumberFormat`.

3. Run the app. Now you should see the formatted price string for subtotal and total. This is much more user-friendly!



4. Test that it works as expected. Test cases like: Order one cupcake, order six cupcakes, order 12 cupcakes. Make sure the price is correctly updated on each screen. It should say **Subtotal \$2.00** for the Flavor and Pickup fragments, and **Total \$2.00** for the order summary. Also, make sure the order summary shows the correct order details.

9. Setup click listeners using listener binding

In this task, you will use listener binding to bind the button click listeners in the fragment classes to the layout.

1. In the layout file `fragment_start.xml`, add a data variable called `startFragment` of the type `com.example.cupcake.StartFragment`. Make sure the package name of the fragment matches with your app's package name.

```
<layout ...>
    <data>
        <variable
            name="startFragment"
            type="com.example.cupcake.StartFragment" />
        </data>
        ...
    <ScrollView ...>
```

2. In `StartFragment.kt`, in `onViewCreated()` method, bind the new data variable to the fragment instance. You can access the fragment instance inside the fragment using `this`

keyword. Remove the `binding?.apply` block and along with the code within. The completed method should look like this.

```
override fun onCreateView(view: View, savedInstanceState: Bundle?) {  
    super.onCreateView(view, savedInstanceState)  
    binding?.startFragment = this  
}
```

3. In `fragment_start.xml`, add event listeners using listener binding to the `onClick` attribute for the buttons, make a call to `orderCupcake()` on `startFragment`, passing in the number of cupcakes.

```
<Button  
    android:id="@+id/order_one_cupcake"  
    android:onClick="@{() -> startFragment.orderCupcake(1)}"  
    ... />  
  
<Button  
    android:id="@+id/order_six_cupcakes"  
    android:onClick="@{() -> startFragment.orderCupcake(6)}"  
    ... />  
  
<Button  
    android:id="@+id/order_twelve_cupcakes"  
    android:onClick="@{() -> startFragment.orderCupcake(12)}"  
    ... />
```

4. Run the app. Notice the button click handlers in the start fragment are working as expected.
5. Similarly add the above data variable in other layouts as well to bind the fragment instance, `fragment_flavor.xml`, `fragment_pickup.xml`, and `fragment_summary.xml`.

In `fragment_flavor.xml`

```
<layout ...>  
    <data>  
        <variable  
            ... />  
        <variable  
            name="flavorFragment"  
            type="com.example.cupcake.FlavorFragment" />  
    </data>  
    <ScrollView ...>
```

In `fragment_pickup.xml`:

```
<layout ...>  
    <data>
```

```

        <variable
            ... />

        <variable
            name="pickupFragment"
            type="com.example.cupcake.PickupFragment" />
    </data>

    <ScrollView ...>

```

In `fragment_summary.xml`:

```

<layout ...>

    <data>
        <variable
            ... />

        <variable
            name="summaryFragment"
            type="com.example.cupcake.SummaryFragment" />
    </data>

    <ScrollView ...>

```

6. In the rest of the fragment classes, in `onViewCreated()` methods, delete the code that manually sets the click listener on the buttons.
7. In the `onViewCreated()` methods bind the fragment data variable with the fragment instance. You will use `this` keyword differently here, because inside the `binding?.apply` block, the keyword `this` refers to the binding instance, not the fragment instance. Use `@` and explicitly specify the fragment class name, for example `this@FlavorFragment`. The completed `onViewCreated()` methods should look as follows:

The `onViewCreated()` method in `FlavorFragment` class should look like this:

```

override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
    super.onViewCreated(view, savedInstanceState)

    binding?.apply {
        lifecycleOwner = viewLifecycleOwner
        viewModel = sharedViewModel
        flavorFragment = this@FlavorFragment
    }
}

```

The `onViewCreated()` method in `PickupFragment` class should look like this:

```

override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
    super.onViewCreated(view, savedInstanceState)

    binding?.apply {

```

```

        lifecycleOwner = viewLifecycleOwner
        viewModel = sharedViewModel
        pickupFragment = this@PickupFragment
    }
}

```

The resulting `onViewCreated()` method in `SummaryFragment` class method should look like this:

```

override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
    super.onViewCreated(view, savedInstanceState)

    binding?.apply {
        lifecycleOwner = viewLifecycleOwner
        viewModel = sharedViewModel
        summaryFragment = this@SummaryFragment
    }
}

```

8. Similarly in the other layout files, add listener binding expressions to the `onClick` attribute for the buttons.

In `fragment_flavor.xml`:

```

<Button
    android:id="@+id/next_button"
    android:onClick="@{() -> flavorFragment.goToNextScreen()}"
    ... />

```

In `fragment_pickup.xml`:

```

<Button
    android:id="@+id/next_button"
    android:onClick="@{() -> pickupFragment.goToNextScreen()}"
    ... />

```

In `fragment_summary.xml`:

```

<Button
    android:id="@+id/send_button"
    android:onClick="@{() -> summaryFragment.sendOrder()}"
    ...>

```

9. Run the app to verify the buttons still work as expected. There should be no visible change in behavior, but now you've used listener bindings to set up the click listeners!

Congratulations on completing this codelab and building out the **Cupcake** app! However, the app is not quite done yet. In the next codelab, you will add a **Cancel** button and modify the backstack. You will also learn what is a backstack and other new topics. See you there!

10. Solution code

The solution code for this codelab is in the project shown below. Use the viewmodel branch to pull or download the code.

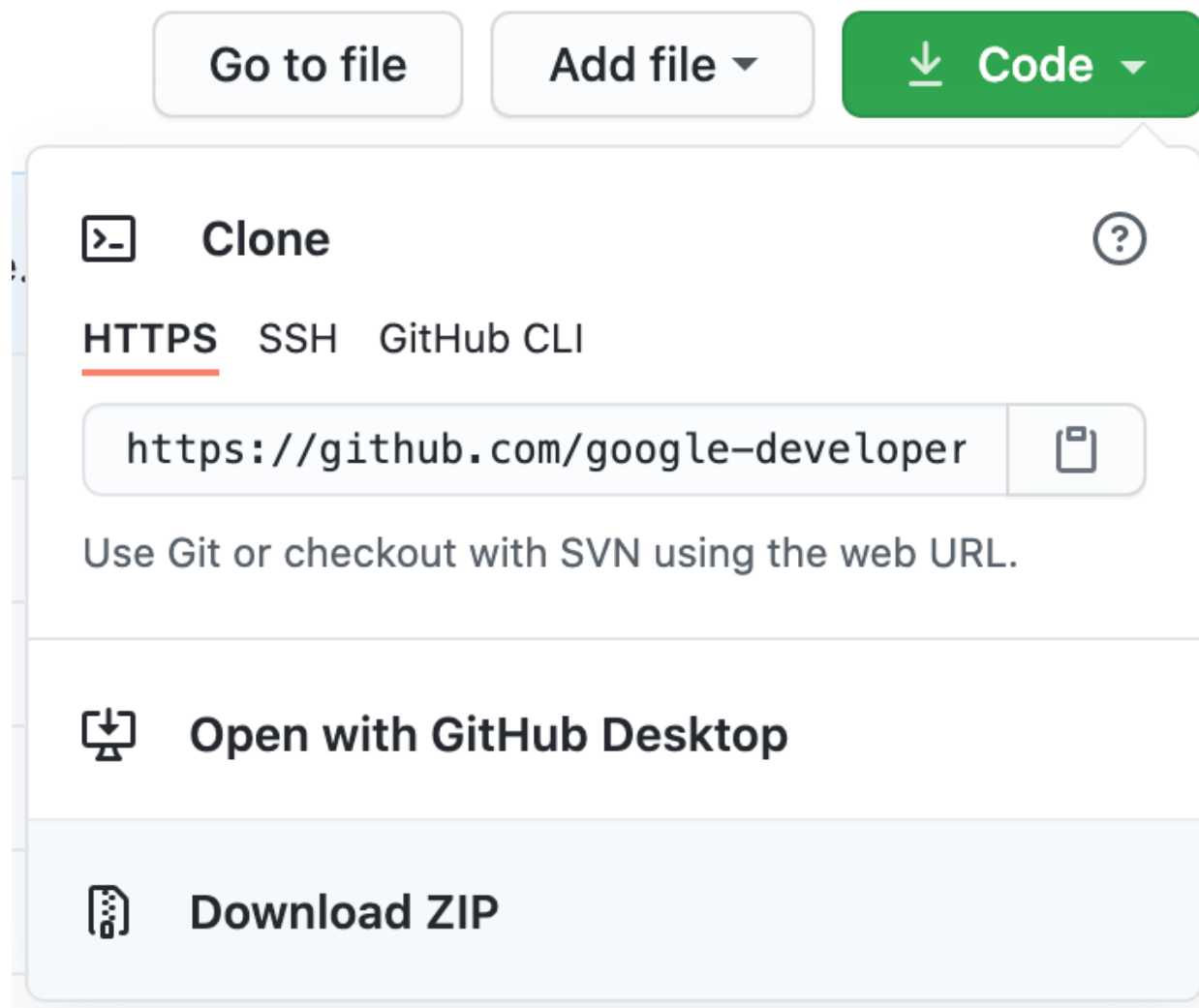
Solution Code URL:

<https://github.com/google-developer-training/android-basics-kotlin-cupcake-app/tree/viewmodel>

To get the code for this codelab and open it in Android Studio, do the following.

Get the code

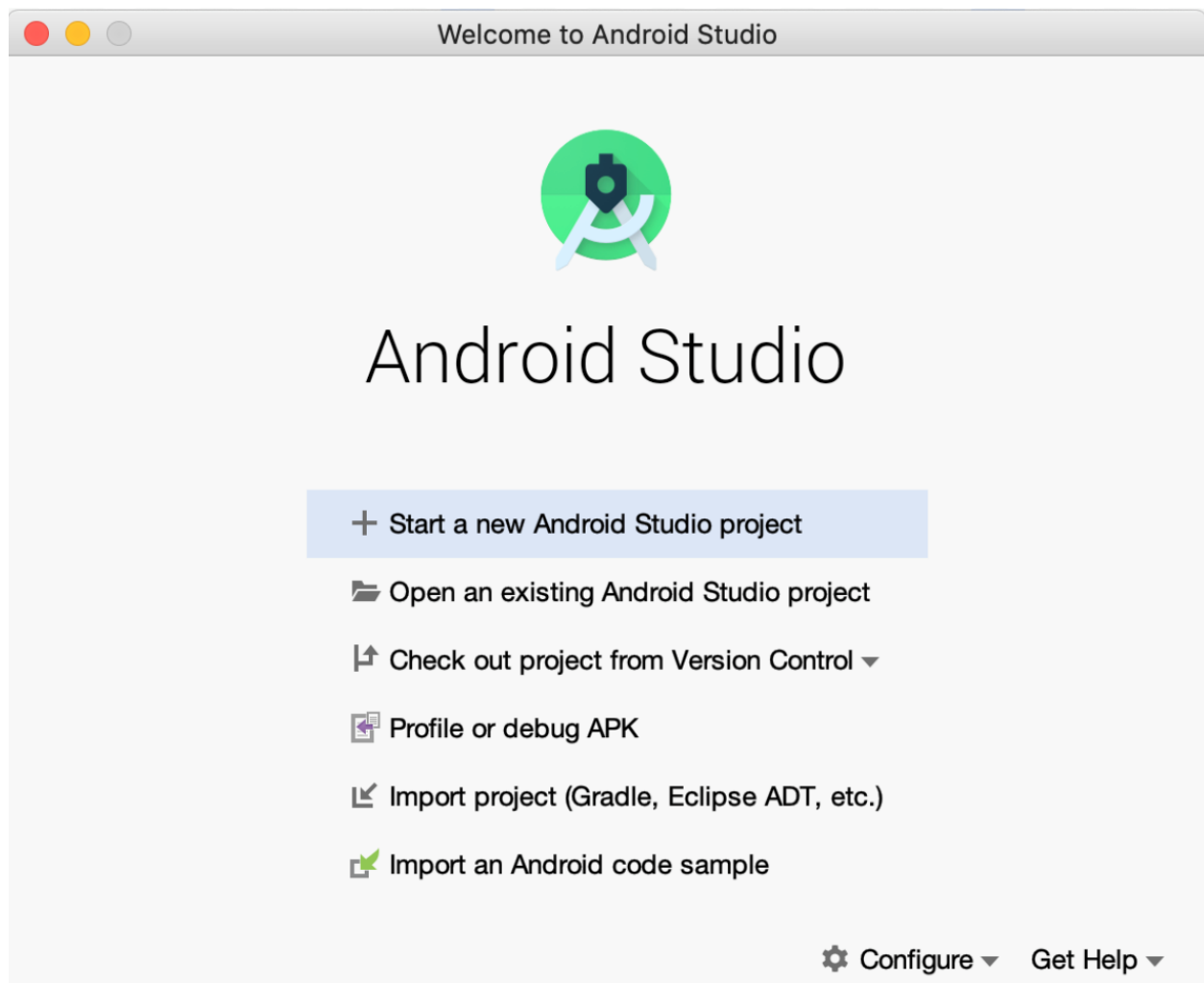
1. Click on the provided URL. This opens the GitHub page for the project in a browser.
2. On the GitHub page for the project, click the **Code** button, which brings up a dialog.



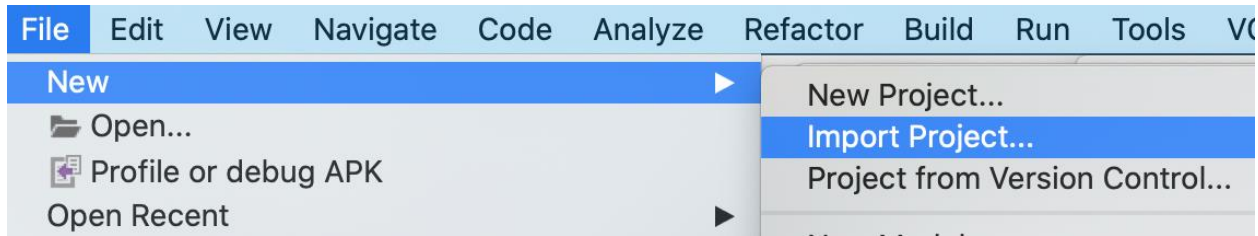
3. In the dialog, click the **Download ZIP** button to save the project to your computer. Wait for the download to complete.
4. Locate the file on your computer (likely in the **Downloads** folder).
5. Double-click the ZIP file to unpack it. This creates a new folder that contains the project files.

Open the project in Android Studio

1. Start Android Studio.
2. In the **Welcome to Android Studio** window, click **Open an existing Android Studio project**.



Note: If Android Studio is already open, instead, select the **File > New > Import Project** menu option.



3. In the **Import Project** dialog, navigate to where the unzipped project folder is located (likely in your **Downloads** folder).
4. Double-click on that project folder.
5. Wait for Android Studio to open the project.



6. Click the **Run** button to build and run the app. Make sure it builds as expected.
7. Browse the project files in the **Project** tool window to see how the app is set-up.

11. Summary

- The [ViewModel](#) is a part of the [Android Architecture Components](#) and the app data saved within the `ViewModel` is retained during configuration changes. To add a `ViewModel` to your app, you create a new class and extend it from the [ViewModel](#) class.
- [Shared ViewModel](#) is used to save the app's data from multiple fragments in a single `ViewModel`. Multiple fragments in the app will access the shared `ViewModel` using their activity scope.
- [LifecycleOwner](#) is a class that has an Android lifecycle, such as an activity or a fragment.
- `LiveData` observer observes the changes to the app's data only if the lifecycle owner is in active states (`STARTED` or `RESUMED`).
- Listener bindings are lambda expressions that run when an event happens such as an `onClick` event. They are similar to method references such as `textView.setOnClickListener(clickListener)` but listener bindings let you run arbitrary data binding expressions.
- The `LiveData` transformation method(s) provides a way to perform data manipulations on the source `LiveData` and return a resulting `LiveData` object.
- Android frameworks provides a class called [SimpleDateFormat](#), a class for formatting and parsing dates in a locale-sensitive manner. It allows for formatting (date → text) and parsing (text → date) dates.

12. Learn more

- [Navigation Component](#)
- [ViewModel Overview](#)
- [Data Binding](#)
- [Layout and binding expressions](#)

- [Transform LiveData](#)
- [SimpleDateFormat](#)
- [apply scope function in Kotlin](#)
- [Compile-time Constants](#)